```
forbid(
    principal in ACME::Employee,
    action
    resource in ACME::Document
)
when { context.device.managed == false };
```

Because `forbid` overrides any matching `permit`, this rule neatly layers on top of discretionary, membership, and relationship policies; if the device isn't managed, the decision is `deny`, no matter what else matches.

9.7 Using policy templates and overrides

Cedar policies don't have to be handwritten individually. Templates let the identity team create a reusable pattern once and then instantiate it multiple times with different parameters. This reduces repetition and keeps policies consistent across teams, products, or customers.

9.7.1 Templates

At ACME, the identity team maintains a document access template that enforces standard owner and viewer rules. When a new Customer Collaboration workspace is created, the service instantiates the template to set up the workspace's owner and reader teams, generating a policy instance specific to that workspace.

```
// Template: document-view-template
permit (
    principal in ?readers_team,
    action == ACME::"doc:view",
    resource in ACME::Document
)
when { resource.owner == ?owner };
```

Template parameters such as `?readers_team` and `?owner` are populated during instantiation. The resulting policy is a standard Cedar policy, fully type-checked and deployed alongside the rest.